

SMAI Mid-Term Examination – Answer Key & Grading Rubric

Q-I: Short Answer Questions [30 min; $10 \times 3 = 30$ points]

Q1.1. A set $\mathcal{X} = \{\mathbf{x}_i\}$ of N samples $\mathbf{x}_i \in \mathbb{R}^d$ from a multivariate Gaussian. $\Sigma = ?$

Answer

First compute the mean: $\boldsymbol{\mu} = \frac{1}{N} \sum_{i=1}^N \mathbf{x}_i$. Then the covariance matrix:

$$\Sigma = \frac{1}{N} \sum_{i=1}^N (\mathbf{x}_i - \boldsymbol{\mu})(\mathbf{x}_i - \boldsymbol{\mu})^T$$

Note: $\frac{1}{N-1}$ (Bessel's correction / unbiased estimator) is also acceptable. In matrix form, if $\bar{X}$ is the $d \times N$ mean-centered data matrix (each column is $\mathbf{x}_i - \boldsymbol{\mu}$), then $\Sigma = \frac{1}{N} \bar{X} \bar{X}^T$.

Grading Rubric

Correct full expression with mean subtraction and outer product	3/3
Correct expression but $\boldsymbol{\mu}$ written but not defined as mean	2.5/3
Wrote $\frac{1}{N} \sum \mathbf{x}_i \mathbf{x}_i^T$ without mean subtraction	1/3
Wrote only the scalar variance formula (not the matrix outer-product form)	0.5/3
Missing transpose (wrote $(\mathbf{x}_i - \boldsymbol{\mu})(\mathbf{x}_i - \boldsymbol{\mu})$ instead of outer product)	1.5/3
Blank / completely wrong	0/3

Q1.2. Transformation $\mathbf{x}' = \mathbf{W}\mathbf{x}$ where $x'^1 = x^1 + x^2$ and $x'^2 = x^1 - x^2$. Write $\mathbf{W}$.

Answer

$$\begin{bmatrix} x'^1 \\ x'^2 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} x^1 \\ x^2 \end{bmatrix} \implies \boxed{\mathbf{W} = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}}$$

Grading Rubric

Correct 2×2 matrix	3/3
Transpose of the correct matrix (rows/columns swapped)	1/3
Correct idea but sign error in one entry	1.5/3
Blank / completely wrong / wrong dimensions	0/3

Q1.3. Data in $\mathbb{R}^2$ with mean $[1, 1]^T$. Make all samples unit norm (L2) and mean zero. Write Python snippet.

Answer

Standard preprocessing: **first center, then normalize.**

```
# X is (N, 2) array
# Step 1: Center (make mean zero)
X = X - np.mean(X, axis=0)      # or: X = X - np.array([1, 1])

# Step 2: Normalize each sample to unit L2 norm
norms = np.linalg.norm(X, axis=1, keepdims=True)
X = X / norms
```

Note: After L2 normalization the mean will no longer be exactly zero. The question asks for both operations; the canonical order is center-then-normalize. Either order is accepted.

Grading Rubric

Both centering and L2 normalization correct with proper axis/- keepdims	3/3
Both steps present but minor syntax issue (e.g., wrong axis) but intent clear	2/3
Only centering correct, normalization missing or wrong	1.5/3
Only normalization correct, centering missing	1/3
Centering with hardcoded [1,1] instead of np.mean – acceptable (mean is given)	3/3
Used sklearn StandardScaler – does not do L2 normalization, wrong	0.5/3
Blank / completely wrong	0/3

Q1.4. KNN: “If we increase K , the decision boundary becomes more and more smooth.” TRUE or FALSE?

Answer

TRUE. Increasing K means each prediction averages over a larger neighborhood, smoothing out local irregularities. Small K (e.g., $K = 1$) overfits to noise and yields highly jagged boundaries.

Grading Rubric

Correct answer: TRUE	3/3
FALSE	0/3
<i>No partial credit. Direct TRUE/FALSE.</i>	

Q1.5. MSE loss $\mathcal{L} = \frac{1}{N} \sum_{i=1}^N (y_i - \mathbf{w}^T \mathbf{x}_i)^2$. Write the gradient descent update for $\mathbf{w}^{k+1}$.

Answer

Gradient: $\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = -\frac{2}{N} \sum_{i=1}^N (y_i - \mathbf{w}^T \mathbf{x}_i) \mathbf{x}_i$. Update rule:

$$\mathbf{w}^{k+1} = \mathbf{w}^k + \frac{2\eta}{N} \sum_{i=1}^N (y_i - (\mathbf{w}^k)^T \mathbf{x}_i) \mathbf{x}_i$$

In matrix form ($X: N \times d$, $\mathbf{y}: N \times 1$): $\mathbf{w}^{k+1} = \mathbf{w}^k + \frac{2\eta}{N} X^T (\mathbf{y} - X \mathbf{w}^k)$.

Grading Rubric

Correct update with correct gradient (sign, factor of 2, $1/N$)	3/3
Correct structure but missing factor of 2	2.5/3
Correct structure but wrong sign (gradient ascent instead of descent)	1/3
Correct gradient but did not write the update equation	1.5/3
Missing $1/N$ but otherwise correct	2/3
Only wrote $\mathbf{w}^{k+1} = \mathbf{w}^k - \eta \nabla \mathcal{L}$ without expanding gradient	1/3
Blank / completely wrong	0/3

Q1.6. Optimal solution to overdetermined homogeneous system $\mathbf{A}\mathbf{x} = \mathbf{0}$ ($\mathbf{A}: m \times n$, $m > n$).

Answer

We seek $\min_{\mathbf{x}} \|\mathbf{A}\mathbf{x}\|^2$ subject to $\|\mathbf{x}\| = 1$. Since $\|\mathbf{A}\mathbf{x}\|^2 = \mathbf{x}^T \mathbf{A}^T \mathbf{A} \mathbf{x}$, this is a Rayleigh quotient minimization over the $n \times n$ PSD matrix $\mathbf{A}^T \mathbf{A}$. The minimum is the eigenvector for the **smallest eigenvalue** of $\mathbf{A}^T \mathbf{A}$.

(d) Eigenvector corresponding to the smallest eigenvalue of $\mathbf{A}^T \mathbf{A}$

Why not (c)? $\mathbf{A}\mathbf{A}^T$ is $m \times m$; its eigenvectors are in $\mathbb{R}^m$, but we need $\mathbf{x} \in \mathbb{R}^n$.

Grading Rubric

Correct: (d)	3/3
Chose (c) – correct idea (smallest eigenvalue) but wrong matrix	1/3
Chose (b) – correct matrix but largest eigenvalue	0.5/3
Chose (a)	0/3

No other partial credit. MCQ.

Q1.7. $A = aI$ (diagonal with all entries a). What is the SVD UDV^T of A ?

Answer

Since $A = aI$ is already diagonal:

$$U = I, \quad D = |a| I, \quad V = \text{sign}(a) I$$

For $a > 0$: simply $A = I \cdot aI \cdot I^T$. Singular values are all $|a|$.

Acceptable short answer: $U = I$, $D = aI$, $V = I$ (assuming $a > 0$). Any valid orthogonal U, V pair satisfying $UDV^T = aI$ with $D = |a|I$ is accepted.

Grading Rubric

$U = I$, $D = aI$ (or $ a I$), $V = I$ (for $a > 0$ case)	3/3
Correct structure, did not address sign (acceptable)	3/3
Only stated singular values are all a without giving U, V	1.5/3
U and V as identity but D wrong	1.5/3
Any valid orthogonal U, V with $D = a I$ such that $UDV^T = aI$	3/3
Blank / completely wrong	0/3

Q1.8. 1D data $\{(+1, +), (+2, +), (+3, -), (+4, -)\}$. SVM decision rule and support vectors?

Answer

Class +: $\{1, 2\}$, Class -: $\{3, 4\}$. Linearly separable. The SVM maximizes margin between the closest opposing points: $x = 2$ (class +) and $x = 3$ (class -).

Support vectors: $x = 2$ (+) and $x = 3$ (-)

Decision boundary at the midpoint: $x = 2.5$.

Decision rule: $f(x) = \text{sign}(-x + 2.5)$, i.e., predict + if $x < 2.5$, predict - if $x > 2.5$.

Grading Rubric

Correct support vectors ($x=2, 3$) and correct boundary ($x=2.5$)	3/3
Correct support vectors but decision boundary missing or slightly off	2/3
Correct boundary but support vectors not identified	1.5/3
Identified all four points as support vectors	1/3
Only boundary correct, support vectors wrong	1/3
Blank / completely wrong	0/3

Q1.9. 100 samples each from $\mathcal{N}(20, \sigma^2)$ and $\mathcal{N}(30, 2\sigma^2)$. Optimal Bayesian threshold?

Answer

(a) closer to 20

Intuitive explanation:

Picture the two bell curves sitting at $\mu_1 = 20$ and $\mu_2 = 30$. The class at 30 has variance $2\sigma^2$ – it is *wider and flatter*. The class at 20 has variance σ^2 – it is *narrower and taller*. Now imagine walking from 20 toward 30. The narrow distribution at 20 drops off *steeply* – its density decays like $e^{-\Delta^2/2\sigma^2}$. Meanwhile, the wide distribution centered at 30, despite being further away, decays *much more slowly* – like $e^{-\Delta^2/4\sigma^2}$ (the denominator is twice as large). Because the wider distribution's tail extends further and decays more gently, it “invades” the territory of the narrow distribution. Its likelihood overtakes the narrow distribution's likelihood at a point that is **closer to 20 than to 30**.

Analogy: Think of two light sources – a bright focused spotlight at position 20 and a dim diffuse lamp at position 30. Even though the spotlight is brighter at its center, its light falls off sharply with distance. The diffuse lamp's light reaches much further. The

point where both sources contribute equal illumination will be *closer to the spotlight*, because its light runs out sooner.

In short: the decision boundary is pulled toward the mean of the *tighter* (faster-decaying) distribution, because the wider distribution's density remains competitive much further from its own mean.

Mathematical verification:

With equal priors ($P(C_1) = P(C_2) = 1/2$), set log-likelihoods equal for $C_1 \sim \mathcal{N}(20, \sigma^2)$ and $C_2 \sim \mathcal{N}(30, 2\sigma^2)$:

$$-\frac{(x-20)^2}{2\sigma^2} - \frac{1}{2}\ln(\sigma^2) = -\frac{(x-30)^2}{4\sigma^2} - \frac{1}{2}\ln(2\sigma^2)$$

The normalization constants give: $-\frac{1}{2}\ln\sigma^2 + \frac{1}{2}\ln(2\sigma^2) = \frac{1}{2}\ln 2$. Multiplying through by $4\sigma^2$:

$$-2(x-20)^2 + (x-30)^2 = -2\sigma^2 \ln 2$$

Expanding: $-x^2 + 20x + 100 = -2\sigma^2 \ln 2$. Solving the quadratic:

$$x^* = 10 + \sqrt{200 + 2\sigma^2 \ln 2}$$

For small to moderate σ :

σ	x^*	Closer to
$\rightarrow 0$	$\rightarrow 24.14$	20
1	24.19	20
2	24.34	20
3	24.58	20
4	24.91	20

The threshold equals exactly 25 at $\sigma_c = \sqrt{25/(2\ln 2)} \approx 4.25$. For $\sigma < \sigma_c \approx 4.25$ the boundary is closer to 20. For very large $\sigma > 4.25$ the normalization-constant effect can push the threshold past 25, but this is a degenerate regime where the two means are separated by less than one standard deviation and the distributions almost completely overlap.

Option (c) is definitively wrong since the threshold depends on σ . Option (d) requires extreme $\sigma > 12$.

Grading Rubric

Chose (a)	3/3
Chose (b) with reasoning about variance shifting boundary	1/3
Chose (c) – wrong (threshold depends on σ)	0/3
Chose (d)	0/3

No partial credit beyond what is listed. Direct MCQ.

Q1.10. (a) K-Means objective function. (b) Does it guarantee a global unique solution?

Answer

(a) K-Means minimizes the Within-Cluster Sum of Squares (WCSS / Inertia):

$$J = \sum_{k=1}^K \sum_{\mathbf{x}_i \in C_k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2$$

where C_k is the set of points in cluster k and $\boldsymbol{\mu}_k = \frac{1}{|C_k|} \sum_{\mathbf{x}_i \in C_k} \mathbf{x}_i$.

(b) **No.** K-Means is **not convex**. The objective involves discrete assignment variables making it NP-hard in general. The algorithm only converges to a **local minimum** that depends on initialization.

Grading Rubric

Part (a):

Correct objective with double sum, distance, and cluster notation	2/3
Correct idea but missing squared norm (wrote $\ \cdot\ $ instead of $\ \cdot\ ^2$)	1/3
Wrong objective entirely	0/3

Part (b):

Correct: No, non-convex, local minimum, initialization-dependent	1/3
Says "No" but no explanation	0.5/3
Says "Yes"	0/3

Q-II: Precise Answers

[30 min; $4 \times 7.5 = 30$ points]

Q2.1. Bag I has w_1 white and b_1 black. Bag II has w_2 white and b_2 black. A ball drawn at random from one bag is white. Probability it came from Bag I?

Answer

Relevant probabilities:

- Priors: $P(\text{Bag I}) = \frac{1}{2}$, $P(\text{Bag II}) = \frac{1}{2}$ (chosen at random)
- Likelihoods: $P(W | \text{Bag I}) = \frac{w_1}{w_1 + b_1}$, $P(W | \text{Bag II}) = \frac{w_2}{w_2 + b_2}$
- Total probability (law of total probability): $P(W) = \frac{1}{2} \cdot \frac{w_1}{w_1 + b_1} + \frac{1}{2} \cdot \frac{w_2}{w_2 + b_2}$

Bayes' theorem:

$$P(\text{Bag I} | W) = \frac{P(W | \text{Bag I}) P(\text{Bag I})}{P(W)} = \frac{\frac{w_1}{w_1 + b_1}}{\frac{w_1}{w_1 + b_1} + \frac{w_2}{w_2 + b_2}}$$

Grading Rubric

All individual probabilities correctly stated (priors, likelihoods, total)	3/7.5
Correct application of Bayes' theorem formula	2/7.5
Correct final simplified expression	2.5/7.5
<i>Deductions:</i>	
Missing or wrong prior (e.g., not 1/2 each)	-1
Wrong likelihood (e.g., $w_1/(w_1 + w_2)$ instead of $w_1/(w_1 + b_1)$)	-2
Bayes formula invoked but with wrong terms	-2
Wrote Bayes formula correctly but did not expand	4/7.5
Only final answer, no intermediate probabilities	4/7.5
Blank / completely wrong	0/7.5

Q2.2. $\kappa(\mathbf{p}, \mathbf{q}) = \phi(\mathbf{p})^T \phi(\mathbf{q})$, $K_{ij} = \kappa(\mathbf{x}_i, \mathbf{x}_j)$. Show (a) K symmetric, (b) K PSD.

Answer

(a) K is symmetric: $K_{ij} = \kappa(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$. Since the dot product of real vectors is commutative: $\phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j) = \phi(\mathbf{x}_j)^T \phi(\mathbf{x}_i) = K_{ji}$. Hence $K_{ij} = K_{ji}$ for all i, j , so $K = K^T$. $\square$

(b) K is PSD: For any $\mathbf{c} = [c_1, \dots, c_N]^T \in \mathbb{R}^N$:

$$\mathbf{c}^T K \mathbf{c} = \sum_i \sum_j c_i c_j K_{ij} = \sum_i \sum_j c_i c_j \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j) = \left\| \sum_i c_i \phi(\mathbf{x}_i) \right\|^2 \geq 0$$

The key step: the double sum factors into a squared norm $\|\mathbf{v}\|^2$ where $\mathbf{v} = \sum_i c_i \phi(\mathbf{x}_i)$, which is always ≥ 0 . $\square$

Alternative: Let Φ have rows $\phi(\mathbf{x}_i)^T$. Then $K = \Phi \Phi^T$, so $\mathbf{c}^T K \mathbf{c} = \|\Phi^T \mathbf{c}\|^2 \geq 0$.

Grading Rubric

Part (a) – Symmetry [3/7.5]:	
Correct proof using commutativity of dot product	3/3
Stated $K_{ij} = K_{ji}$ without showing why	2/3
Stated “ K is symmetric” without justification	0.5/3
Part (b) – PSD [4.5/7.5]:	
Full proof: arbitrary $\mathbf{c}$, expanded, reached $\ \cdot\ ^2 \geq 0$	4.5/4.5
Correct setup and expansion but did not reach $\ \cdot\ ^2$ form	2.5/4.5
Wrote $K = \Phi \Phi^T$ and claimed PSD without $\mathbf{c}^T K \mathbf{c} \geq 0$	2/4.5
Only stated eigenvalues are non-negative without proof	1/4.5
Blank / completely wrong	0/4.5

Q2.3. $f(w) = w^2 + 2w + 1$, $w^0 = 5$, $\eta = 0.1$. (a) w^1, w^2 . (b) Better η . (c) Optimal one-step η .

Answer

Note: $f(w) = (w + 1)^2$, minimum at $w^* = -1$. Gradient: $f'(w) = 2w + 2$.

(a) Two iterations:

$$w^1 = w^0 - \eta f'(w^0) = 5 - 0.1(2 \cdot 5 + 2) = 5 - 1.2 = \boxed{3.8}$$

$$w^2 = w^1 - \eta f'(w^1) = 3.8 - 0.1(2 \cdot 3.8 + 2) = 3.8 - 0.96 = \boxed{2.84}$$

(b) With $\eta = 0.1$, after 2 iterations we are at 2.84, still far from -1 . Any $\eta \in (0.1, 1)$ converges faster, e.g. $\eta = 0.3$: $w^1 = 5 - 0.3 \times 12 = 1.4$, $w^2 = 1.4 - 0.3 \times 4.8 = -0.04$.

(c) Need $w^1 = w^* = -1$: $5 - \eta \times 12 = -1 \Rightarrow \eta = 6/12 = \boxed{0.5}$

Note: for a quadratic, $\eta^* = 1/f''(w) = 1/2$ (Newton's method step size).

Grading Rubric

Part (a) [3/7.5]:

Both $w^1 = 3.8$ and $w^2 = 2.84$ correct with work shown	3/3
w^1 correct, w^2 has arithmetic error	2/3
Correct method but both have arithmetic errors	1/3
Wrong update formula (e.g., added gradient)	0.5/3

Part (b) [1.5/7.5]:

Any valid $\eta > 0.1$ with brief justification or numerical check	1.5/1.5
Valid η but no justification	1/1.5
$\eta \geq 1$ (diverges for this problem)	0/1.5

Part (c) [3/7.5]:

$\eta = 0.5$ with derivation	3/3
$\eta = 0.5$ without showing work	2/3
Correct setup ($5 - 12\eta = -1$) but arithmetic error	1.5/3
Blank / wrong	0/3

Q2.4. Linear regression on $\{(1, 2), (2, 3), (3, 4), (4, 5)\}$. (i) Guess $\mathbf{w}$. (ii) Python for general linear regression.

Answer

Part (i): Observing the data: $y = x + 1$ for every sample.

$$w_1 = 1, \quad w_2 = 1 \quad \Rightarrow \quad \mathbf{w} = [1, 1]^T$$

Part (ii): For $y_i = \theta^T \mathbf{x}_i + b$, augment $\mathbf{x}_i$ with 1 to absorb bias: $\tilde{\mathbf{x}}_i = [\mathbf{x}_i; 1]$, $\tilde{\boldsymbol{\theta}} = [\boldsymbol{\theta}; b]$. Closed-form: $\tilde{\boldsymbol{\theta}} = (\tilde{X}^T \tilde{X})^{-1} \tilde{X}^T \mathbf{y}$.

```
import numpy as np
# X: (N, d) data matrix, y: (N,) targets
N, d = X.shape
X_aug = np.hstack([X, np.ones((N, 1))]) # (N, d+1)
theta_aug = np.linalg.inv(X_aug.T @ X_aug) @ X_aug.T @ y # (d+1,)
theta, b = theta_aug[:d], theta_aug[d]
```

Alternative (gradient descent):

```
import numpy as np
N, d = X.shape
X_aug = np.hstack([X, np.ones((N, 1))]) # (N, d+1)
w = np.zeros(d + 1)
eta = 0.01
for _ in range(1000):
    grad = -(2.0 / N) * (X_aug.T @ (y - X_aug @ w)) # (d+1,)
    w = w - eta * grad
theta, b = w[:d], w[d]
```


Grading Rubric

Part (i) [2/7.5]:

Correct: $w_1 = 1, w_2 = 1$	2/2
One of w_1, w_2 correct	1/2
Wrong guess	0/2

Part (ii) [5.5/7.5]:

Closed-form approach:

Correct augmentation of X with ones column	1/5.5
Correct normal equation $(X^T X)^{-1} X^T y$ or <code>np.linalg.lstsq</code>	2/5.5
Correct extraction of θ and b	1/5.5
Correct matrix dimensions throughout	1.5/5.5

Common deductions:

Missing bias handling (no augmentation, no separate b)	-2
Incompatible matrix dimensions	-1.5
Pseudocode instead of Python (clear and correct)	-0.5
Used <code>sklearn</code> API (explicitly disallowed)	max 1/5.5

Q-III: Detailed Answers

[30 min; $2 \times 15 = 30$ points]

Q3.1. K-Means on 10 samples in $\mathbb{R}^3$, 3 clusters, 3 iterations.

Answer

(i) Steps of K-Means:

1. **Initialize:** assign each point to one of K clusters (given).
2. **Update centroids:** $\mu_k = \frac{1}{|C_k|} \sum_{\mathbf{x}_i \in C_k} \mathbf{x}_i$
3. **Reassign:** $C(\mathbf{x}_i) = \arg \min_k \|\mathbf{x}_i - \mu_k\|^2$
4. **Repeat** steps 2–3 until no reassignment.

(ii) Numerical computation:

Data: $\mathbf{x}_1 = [1, 1, 1]^T$, $\mathbf{x}_2 = [-1, -1, -1]^T$, $\mathbf{x}_3 = [0, 0, 0]^T$, $\mathbf{x}_4 = [5, 5, 0]^T$, $\mathbf{x}_5 = [4, 4, 0]^T$, $\mathbf{x}_6 = [6, 6, 0]^T$, $\mathbf{x}_7 = [6, 4, 0]^T$, $\mathbf{x}_8 = [4, 6, 0]^T$, $\mathbf{x}_9 = [0, -6, -4]^T$, $\mathbf{x}_{10} = [0, -4, -6]^T$

Initial: $C_1 = \{\mathbf{x}_1, \mathbf{x}_4, \mathbf{x}_7, \mathbf{x}_{10}\}$, $C_2 = \{\mathbf{x}_2, \mathbf{x}_5, \mathbf{x}_8\}$, $C_3 = \{\mathbf{x}_3, \mathbf{x}_6, \mathbf{x}_9\}$.

ITERATION 1 – Compute Centroids:

$$\begin{aligned}\mu_1 &= \frac{1}{4}([1, 1, 1] + [5, 5, 0] + [6, 4, 0] + [0, -4, -6]) = \frac{1}{4}[12, 6, -5] = [3, 1.5, -1.25] \\ \mu_2 &= \frac{1}{3}([-1, -1, -1] + [4, 4, 0] + [4, 6, 0]) = \frac{1}{3}[7, 9, -1] = [2.33, 3, -0.33] \\ \mu_3 &= \frac{1}{3}([0, 0, 0] + [6, 6, 0] + [0, -6, -4]) = \frac{1}{3}[6, 0, -4] = [2, 0, -1.33]\end{aligned}$$

ITERATION 1 – Reassign (squared distances $\|\mathbf{x}_i - \mu_k\|^2$):

Point	$d^2(\mu_1)$	$d^2(\mu_2)$	$d^2(\mu_3)$	Assign
$\mathbf{x}_1 = [1, 1, 1]$	9.31	7.56	7.44	C_3
$\mathbf{x}_2 = [-1, -1, -1]$	22.31	27.56	10.11	C_3
$\mathbf{x}_3 = [0, 0, 0]$	12.81	14.56	5.78	C_3
$\mathbf{x}_4 = [5, 5, 0]$	17.81	11.22	35.78	C_2
$\mathbf{x}_5 = [4, 4, 0]$	8.81	3.89	21.78	C_2
$\mathbf{x}_6 = [6, 6, 0]$	30.81	22.56	53.78	C_2
$\mathbf{x}_7 = [6, 4, 0]$	16.81	14.56	33.78	C_2
$\mathbf{x}_8 = [4, 6, 0]$	22.81	11.89	41.78	C_2
$\mathbf{x}_9 = [0, -6, -4]$	72.81	99.89	47.11	C_3
$\mathbf{x}_{10} = [0, -4, -6]$	61.81	86.56	41.78	C_3

After Iteration 1: $C_1 = \{\}$ (empty!), $C_2 = \{\mathbf{x}_4, \mathbf{x}_5, \mathbf{x}_6, \mathbf{x}_7, \mathbf{x}_8\}$, $C_3 = \{\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3, \mathbf{x}_9, \mathbf{x}_{10}\}$.

Handling the empty cluster: Re-seed μ_1 with the point farthest from its assigned centroid. Among C_3 points, $\mathbf{x}_9$ is farthest from μ_3 ($d^2 = 47.11$); we re-seed with $\mathbf{x}_9 = [0, -6, -4]$.

(If a student proceeds with $K=2$ after noting the empty cluster, this is also acceptable.)

Revised: $C_1 = \{\mathbf{x}_9\}$, $C_2 = \{\mathbf{x}_4, \mathbf{x}_5, \mathbf{x}_6, \mathbf{x}_7, \mathbf{x}_8\}$, $C_3 = \{\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3, \mathbf{x}_{10}\}$.

ITERATION 2 – Compute Centroids:

$$\mu_1 = [0, -6, -4] \text{ (singleton)}$$

$$\mu_2 = \frac{1}{5}[25, 25, 0] = [5, 5, 0]$$

$$\mu_3 = \frac{1}{4}([1, 1, 1] + [-1, -1, -1] + [0, 0, 0] + [0, -4, -6]) = \frac{1}{4}[0, -4, -6] = [0, -1, -1.5]$$

ITERATION 2 – Reassign:

Point	$d^2(\mu_1)$	$d^2(\mu_2)$	$d^2(\mu_3)$	Assign
$\mathbf{x}_1$	75	33	11.25	C_3
$\mathbf{x}_2$	35	73	1.25	C_3
$\mathbf{x}_3$	52	50	3.25	C_3
$\mathbf{x}_4$	162	0	63.25	C_2
$\mathbf{x}_5$	132	2	43.25	C_2
$\mathbf{x}_6$	196	2	87.25	C_2
$\mathbf{x}_7$	152	2	63.25	C_2
$\mathbf{x}_8$	176	2	67.25	C_2
$\mathbf{x}_9$	0	162	31.25	C_1
$\mathbf{x}_{10}$	8	142	29.25	C_1

After Iteration 2: $C_1 = \{\mathbf{x}_9, \mathbf{x}_{10}\}$, $C_2 = \{\mathbf{x}_4, \mathbf{x}_5, \mathbf{x}_6, \mathbf{x}_7, \mathbf{x}_8\}$, $C_3 = \{\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3\}$.

ITERATION 3 – Compute Centroids:

$$\mu_1 = \frac{1}{2}([0, -6, -4] + [0, -4, -6]) = [0, -5, -5]$$

$$\mu_2 = [5, 5, 0] \text{ (unchanged)}$$

$$\mu_3 = \frac{1}{3}([1, 1, 1] + [-1, -1, -1] + [0, 0, 0]) = [0, 0, 0]$$

ITERATION 3 – Reassign:

Point	$d^2(\mu_1)$	$d^2(\mu_2)$	$d^2(\mu_3)$	Assign
$\mathbf{x}_1$	73	33	3	C_3
$\mathbf{x}_2$	33	73	3	C_3
$\mathbf{x}_3$	50	50	0	C_3
$\mathbf{x}_4$	150	0	50	C_2
$\mathbf{x}_5$	122	2	32	C_2
$\mathbf{x}_6$	182	2	72	C_2
$\mathbf{x}_7$	142	2	52	C_2
$\mathbf{x}_8$	162	2	52	C_2
$\mathbf{x}_9$	2	162	52	C_1
$\mathbf{x}_{10}$	2	142	52	C_1

No change $\Rightarrow$ converged. Final clusters:

$$C_1 = \{\mathbf{x}_9, \mathbf{x}_{10}\}, \quad \boldsymbol{\mu}_1 = [0, -5, -5]^T$$

$$C_2 = \{\mathbf{x}_4, \mathbf{x}_5, \mathbf{x}_6, \mathbf{x}_7, \mathbf{x}_8\}, \quad \boldsymbol{\mu}_2 = [5, 5, 0]^T$$

$$C_3 = \{\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3\}, \quad \boldsymbol{\mu}_3 = [0, 0, 0]^T$$

Grading Rubric

Part (i) – Algorithm steps [5/15]:

Correct equations/description of both steps (update centroids + reassign) 4.5/5

Continue until convergence 0.5/5

No equation but description is correct 3.5/5

Part (ii) – Numerical computation [10/15]:

Correct centroids (each iteration) 1/10

Correct reassignment with distances shown 2/10

One correct distance computation 1/10

Q3.2. PCA / Eigenface for face recognition. 200 images (100×100). Classify test image Q as “A” or “B” using KNN ($K=5$).

Answer

Setup: Each 100×100 image is reshaped into a 10000×1 vector. Let X be the 200×10000 data matrix (rows = flattened images). First 100 rows from “A”, next 100 from “B”.

Step (i): Mean and Covariance

$$\boldsymbol{\mu} = \frac{1}{N} \sum_{i=1}^N \mathbf{x}_i \in \mathbb{R}^{10000}, \quad \bar{X} = X - \mathbf{1}\boldsymbol{\mu}^T \text{ (centered, } 200 \times 10000\text{)}.$$

$$\text{Covariance: } C = \frac{1}{N} \bar{X}^T \bar{X} \in \mathbb{R}^{10000 \times 10000}.$$

```
import numpy as np
# X: (200, 10000)
mu = np.mean(X, axis=0) # (10000,)
X_centered = X - mu # (200, 10000)
# Use the small-matrix trick (200 x 200 instead of 10000 x 10000)
C_small = (1.0 / N) * X_centered @ X_centered.T # (200, 200)
```

Step (ii): Eigenvectors and Selection

Since $10000 \gg 200$, we use the PCA trick: if $\mathbf{v}$ is an eigenvector of $\bar{X}\bar{X}^T$ (200×200) with eigenvalue λ , then $\mathbf{u} = \bar{X}^T \mathbf{v}$ is an eigenvector of $\bar{X}^T \bar{X}$ (10000×10000) with the same λ .

Choose top d eigenvectors capturing $\geq 95\%$ variance: $d = \min\{k : \sum_{i=1}^k \lambda_i / \sum_{i=1}^N \lambda_i \geq 0.95\}$.

```
eigenvalues, eigvecs_small = np.linalg.eigh(C_small)
idx = np.argsort(eigenvalues)[::-1]
eigenvalues = eigenvalues[idx]
eigvecs_small = eigvecs_small[:, idx]

# Convert to full eigenvectors
eigvecs_full = X_centered.T @ eigvecs_small          # (10000, 200)
norms = np.linalg.norm(eigvecs_full, axis=0, keepdims=True)
eigvecs_full = eigvecs_full / norms                  # normalize

# Choose top d by cumulative variance
cumvar = np.cumsum(eigenvalues) / np.sum(eigenvalues)
d = np.searchsorted(cumvar, 0.95) + 1
W = eigvecs_full[:, :d]                             # (10000, d)
```

Step (iii): Dimensionality Reduction

Project each centered sample: $\mathbf{z}_i = W^T(\mathbf{x}_i - \boldsymbol{\mu}) \in \mathbb{R}^d$.

```
Z_train = X_centered @ W          # (200, d)
```

Step (iv): KNN Classification of Q

Flatten Q, center with *training mean*, project, find 5 nearest neighbors, majority vote.

```
q = Q.flatten()                  # (10000,)
z_q = (q - mu) @ W                # (d,)
distances = np.linalg.norm(Z_train - z_q, axis=1) # (200,)
nn_idx = np.argsort(distances)[:5]
y = np.array([0]*100 + [1]*100) # labels
nn_labels = y[nn_idx]
prediction = 'A' if np.sum(nn_labels == 0) > np.sum(nn_labels ==
1) else 'B'
```

Grading Rubric

Step (i) – Mean & Covariance [3/15]:

Correct mean + centering (equation and/or code)	1.5/3
Correct covariance expression or small-matrix trick	1.5/3

Step (ii) – Eigenvectors [4/15]:

Correct eigendecomposition (small or full matrix)	1.5/4
Conversion from small eigenvectors to full (if trick used)	1/4
Selection criterion for d (variance threshold or explicit choice)	1/4
Normalization of eigenvectors	0.5/4

Step (iii) – Dim. Reduction [3/15]:

Correct projection equation $\mathbf{z}_i = W^T \bar{\mathbf{x}}_i$	1.5/3
Correct code with proper dimensions	1.5/3

Step (iv) – KNN Classification [5/15]:

Flatten, center (with training mean), and project Q correctly	1.5/5
Distance computation in projected space	1/5
KNN with $K=5$ and majority vote	1.5/5
Code clarity and dimension correctness	1/5

Common deductions:

Did not center test image using <i>training</i> mean	−1.5
Wrong projection direction ($W\mathbf{x}$ vs. $W^T\mathbf{x}$)	−1
Missing flattening of 100×100 to 10000×1	−0.5
Pseudocode instead of Python (clear and correct)	−1
KNN on original space instead of projected space	−2
Dimension errors in matrix multiplications	−1 per error